Glosario de Deep Learning

Módulo 2 — Diplomatura en IA · UTN 2026

65 términos · 10 categorías · Para consulta durante toda la cursada

Este glosario reúne los términos fundamentales del Módulo 2. Cada entrada incluye el nombre en español, su equivalente en inglés (el idioma de la bibliografía y las librerías), una definición concisa y un ejemplo concreto del contexto de la cursada.

Conceptos Fundamentales

Red Neuronal Artificial

Artificial Neural Network (ANN)

Sistema computacional inspirado en el cerebro humano, formado por capas de neuronas artificiales interconectadas que aprenden a partir de datos.

Ej: una red que aprende a distinguir fotos de perros y gatos mirando miles de ejemplos.

Neurona Artificial

Artificial Neuron / Perceptrón

Unidad básica de una red neuronal. Recibe entradas, las multiplica por pesos, suma los resultados, le suma un sesgo y aplica una función de activación.

Ej: una neurona que recibe el área y el precio de un barrio para predecir el valor de un departamento.

Perceptrón

Perceptron

El modelo más simple de neurona artificial, propuesto por Rosenblatt en 1957. Clasifica datos linealmente separables usando una función escalón.

Ej: determinar si un email es spam (1) o no (0) basándose en ciertas palabras clave.

Deep Learning

Deep Learning / Aprendizaje Profundo

Rama del Machine Learning que usa redes neuronales con muchas capas (profundas). Las capas intermedias aprenden representaciones cada vez más abstractas de los datos.

Ej: las primeras capas de una red detectan bordes en una imagen; las últimas capas reconocen caras.

Aprendizaje Supervisado

Supervised Learning

Paradigma de entrenamiento donde el modelo aprende de ejemplos etiquetados: para cada entrada, se conoce la salida correcta.

Ej: entrenar un modelo con fotos de radiografías ya clasificadas como 'sano' o 'enfermo'.

Parámetro

Parameter / Weight

Valor numérico interno de una red neuronal (peso o sesgo) que se ajusta durante el entrenamiento. Define el comportamiento del modelo.

Ej: GPT-4 tiene estimados 1.8 billones de parámetros; MobileNetV2 tiene 2.2 millones.

Hiperparámetro

Hyperparameter

Valor de configuración del entrenamiento que se define antes de entrenar, no se aprende automáticamente. Controla cómo aprende el modelo.

Ej: learning rate, número de épocas, tamaño del batch, cantidad de capas.

Inferencia

Inference

Proceso de usar un modelo ya entrenado para hacer predicciones sobre datos nuevos, sin modificar los parámetros.

Ej: cuando dibujás un número en el canvas del notebook y la red predice qué dígito es.

Arquitectura de la Red

Capa de Entrada
Input Layer

Primera capa de la red. Recibe los datos crudos y los pasa a la siguiente capa. No realiza ninguna transformación.

Ej: en MNIST, la capa de entrada tiene 784 neuronas (una por cada píxel del 28x28).

Capa Oculta
Hidden Layer

Capas intermedias entre la entrada y la salida. Aprenden representaciones internas de los datos. Más capas = red más profunda = mayor capacidad.

Ej: Dense(128, activation='relu') en Keras crea una capa oculta con 128 neuronas.

Capa de Salida
Output Layer

Última capa de la red. Produce la predicción final. Su tamaño y activación dependen del problema.

Ej: 10 neuronas con Softmax para clasificar dígitos del 0 al 9; 1 neurona con Sigmoid para clasificación binaria.

Capa Densa
Dense Layer / Fully Connected Layer

Tipo de capa donde cada neurona está conectada a todas las neuronas de la capa anterior. Es la capa más básica de una red neuronal.

Ej: layers.Dense(64, activation='relu') en TensorFlow/Keras.

GlobalAveragePooling
Global Average Pooling

Operación que toma el promedio espacial de un mapa de características. Reduce una salida 3D (alto x ancho x canales) a un vector 1D.

Ej: en Transfer Learning, convierte la salida (7x7x1280) de MobileNetV2 en un vector de 1280 valores.

Arquitectura
Architecture

Estructura completa de una red neuronal: cantidad de capas, tipo de capas, conexiones entre ellas y funciones de activación usadas.

Ej: la arquitectura de nuestra red MNIST es 784 → Dense(128) → Dense(64) → Dense(10).

Entrenamiento

Epoch / Época
Epoch

Una pasada completa por todo el dataset de entrenamiento. El modelo actualiza sus pesos después de procesar cada batch.

Ej: entrenar 10 épocas con 1000 imágenes significa que el modelo vio esas 1000 imágenes 10 veces.

Batch / Lote
Batch / Mini-batch

Subconjunto del dataset que se procesa junto antes de actualizar los pesos. Permite entrenar con datasets grandes que no caben en memoria.

Ej: batch_size=32 significa que el modelo procesa 32 imágenes, calcula el gradiente y actualiza los pesos.

Forward Pass

Forward Pass / Propagación hacia adelante

Proceso de pasar los datos de entrada a través de todas las capas de la red para obtener una predicción.

Ej: una imagen de 784 píxeles entra por la capa de entrada y viaja capa por capa hasta producir 10 probabilidades.

Backpropagation

Backpropagation / Retropropagación

Algoritmo que calcula cuánto contribuyó cada peso al error total, propagando el gradiente desde la salida hacia la entrada.

Ej: si la red predijo '3' cuando era '7', backprop calcula cuánto ajustar cada uno de los 109.000 pesos.

Gradiente Descendente

Gradient Descent

Algoritmo de optimización que ajusta los pesos en la dirección que reduce el error, paso a paso, siguiendo la pendiente de la función de pérdida.

Ej: como una pelota que rueda por una colina buscando el punto más bajo (el mínimo de la pérdida).

Learning Rate / Tasa de Aprendizaje

Learning Rate (lr)

Hiperparámetro que controla el tamaño del paso en cada actualización de pesos. Demasiado alto = diverge; demasiado bajo = aprende muy lento.

Ej: lr=0.001 es un valor típico para Adam. En fine-tuning de Transfer Learning se usa lr=1e-5 para no destruir los pesos pre-entrenados.

Optimizador

Optimizer

Algoritmo que define cómo se actualizan los pesos usando el gradiente calculado por backpropagation.

Ej: Adam es el optimizador más usado. SGD (Stochastic Gradient Descent) es el más clásico.

Adam

Adam (Adaptive Moment Estimation)

Optimizador que combina dos técnicas: momentum (acumula velocidad) y RMSProp (ajusta el lr por parámetro). Es el estándar para la mayoría de los problemas.

Ej: keras.optimizers.Adam(learning_rate=0.001).

Funciones de Activación

Función de Activación

Activation Function

Función no lineal que se aplica a la salida de una neurona. Sin ellas, la red sería equivalente a una simple operación lineal.

Ej: sin activaciones, 10 capas densas seguidas serían equivalentes a una sola capa.

ReLU

Rectified Linear Unit (ReLU)

Activación que devuelve 0 si la entrada es negativa, o la entrada misma si es positiva. f(x) = max(0, x). Es la más usada en capas ocultas.

Ej: si una neurona recibe -3, ReLU devuelve 0. Si recibe 5, devuelve 5.

Sigmoid

Sigmoid / Sigmoide

Activación que aplana cualquier valor al rango (0, 1). Útil para predicciones de probabilidad binaria. f(x) = 1 / (1 + e^{-x}).

Ej: se usa en la capa de salida para clasificación binaria (spam vs. no spam).

Softmax

Softmax

Activación que convierte un vector de valores en probabilidades que suman exactamente 1. Se usa en clasificación multiclase.

Ej: para 10 dígitos, Softmax convierte [2.1, 0.3, -1.2, ...] en [0.72, 0.05, 0.01, ...]. El mayor es la predicción.

Tanh

Hyperbolic Tangent (Tanh)

Activación similar a Sigmoid pero con rango (-1, 1). Antes de ReLU era la más popular para capas ocultas.

Ej: útil cuando los datos están centrados en cero. Hoy ReLU la reemplaza en casi todos los casos.

Funciones de Pérdida y Evaluación

Función de Pérdida

Loss Function / Cost Function

Mide qué tan mal está prediciendo el modelo comparando su salida con la respuesta correcta. El entrenamiento busca minimizarla.

Ej: si la red predijo 0.2 de probabilidad para la clase correcta, la pérdida es alta; si predijo 0.9, es baja.

Cross-Entropy

Cross-Entropy Loss

Función de pérdida estándar para clasificación. Penaliza más los errores seguros (alta confianza en la respuesta equivocada).

Ej: sparse_categorical_crossentropy para multiclase; binary_crossentropy para binario.

MSE

Mean Squared Error (MSE)

Función de pérdida para regresión. Calcula el promedio de las diferencias al cuadrado entre predicción y valor real.

Ej: predecir precios de casas: si el real es $200k y predijo $180k, el error es (200-180)^2 = 400.

Accuracy / Precisión

Accuracy

Porcentaje de predicciones correctas sobre el total. La métrica de evaluación más intuitiva para clasificación.

Ej: accuracy=0.98 en MNIST significa que el modelo acierta 98 de cada 100 dígitos.

Overfitting / Sobreajuste

Overfitting

El modelo memoriza el dataset de entrenamiento en vez de aprender patrones generales. Funciona bien en train pero mal en datos nuevos.

Ej: accuracy de train = 99%, accuracy de validación = 60%. El modelo aprendió de memoria.

Underfitting / Subajuste

Underfitting

El modelo es demasiado simple para capturar los patrones del dataset. Funciona mal tanto en train como en validación.

Ej: accuracy de train = 55%, accuracy de validación = 53%. El modelo no aprendió nada útil.

Matriz de Confusión

Confusion Matrix

Tabla que muestra cuántas predicciones correctas e incorrectas hizo el modelo por cada clase.

Ej: fila 7 / columna 1 = 3 veces predijo '1' cuando era '7'. Los valores en la diagonal son correctos.

Regularización

Regularización

Regularization

Conjunto de técnicas para evitar el overfitting, penalizando la complejidad del modelo o agregando ruido durante el entrenamiento.

Ej: Dropout, Batch Normalization y Early Stopping son las tres técnicas de regularización más usadas.

Dropout

Dropout

Técnica que 'apaga' aleatoriamente un porcentaje de neuronas en cada paso de entrenamiento, forzando a la red a aprender representaciones más robustas.

Ej: Dropout(0.3) apaga el 30% de las neuronas en cada batch. En inferencia, todas están activas.

Batch Normalization

Batch Normalization

Normaliza las activaciones de cada capa dentro de un batch, estabilizando el entrenamiento y permitiendo learning rates más altos.

Ej: layers.BatchNormalization() entre capas densas acelera el entrenamiento hasta 10x en redes profundas.

Early Stopping

Early Stopping

Detiene el entrenamiento cuando la métrica de validación deja de mejorar, evitando que el modelo memorice el dataset.

Ej: EarlyStopping(patience=5) para el entrenamiento si la validación no mejora en 5 épocas seguidas.

Data Augmentation

Data Augmentation

Genera variantes artificiales del dataset aplicando transformaciones (rotación, espejo, zoom) para simular más datos de entrenamiento.

Ej: con 20 fotos de gatos, Data Augmentation puede generar cientos de variantes distintas para entrenar.

L1 / L2 Regularization

L1 / L2 Regularization

Técnicas que agregan una penalización a los pesos grandes en la función de pérdida. L1 produce pesos dispersos; L2 pesos pequeños.

Ej: kernel_regularizer=keras.regularizers.l2(0.01) en una capa Dense.

Transfer Learning

Transfer Learning

Transfer Learning / Aprendizaje por Transferencia

Técnica que reutiliza un modelo entrenado en un problema grande como punto de partida para un problema más pequeño, ahorrando datos y tiempo.

Ej: usar MobileNetV2 entrenado con 14M imágenes para clasificar solo gatos y perros con 20 fotos propias.

Modelo Pre-entrenado

Pre-trained Model

Red neuronal que fue entrenada en un dataset grande y cuyos pesos están disponibles para ser reutilizados.

Ej: MobileNetV2, ResNet50, VGG16, BERT, GPT son todos modelos pre-entrenados de uso libre.

Fine-tuning

Fine-tuning / Ajuste fino

Segunda fase del Transfer Learning: se descongelan las últimas capas del modelo base y se continúa entrenando con un learning rate muy bajo.

Ej: descongelar las últimas 20 capas de MobileNetV2 con lr=1e-5 para ajustarlas a tus imágenes.

Feature Extraction

Feature Extraction / Extracción de características

Primera fase del Transfer Learning: se congela toda la base pre-entrenada y solo se entrenan las capas nuevas agregadas encima.

Ej: congelar MobileNetV2 y entrenar solo Dense(128) + Dense(N_clases) con tus fotos.

MobileNetV2

MobileNetV2

Arquitectura de red convolucional de Google, diseñada para ser liviana y eficiente. Entrenada en ImageNet (14M imágenes, 1000 clases).

Ej: tiene 2.2M de parámetros y pesa solo 14MB. Corre en tiempo real en un celular.

ImageNet

ImageNet

Dataset de 14 millones de imágenes etiquetadas en 1000 categorías, usado como benchmark estándar para redes de visión por computadora.

Ej: los modelos pre-entrenados 'en ImageNet' aprendieron a reconocer animales, objetos, comida, vehículos, etc.

Congelado / Descongelado

Frozen / Unfrozen

Un layer congelado no actualiza sus pesos durante el entrenamiento. Uno descongelado sí lo hace.

Ej: base_model.trainable = False congela MobileNetV2. Solo las capas nuevas se entrenan.

Procesamiento de Imágenes

CNN

Convolutional Neural Network (CNN)

Tipo de red neuronal especializada en imágenes. Usa convoluciones para detectar patrones locales (bordes, texturas) independientemente de su posición.

Ej: YOLO, ResNet, MobileNetV2 son todas CNNs.

Convolución

Convolution

Operación matemática que desliza un filtro (kernel) sobre la imagen y multiplica los valores, detectando patrones como bordes o texturas.

Ej: un filtro de 3x3 detecta bordes verticales; otro detecta bordes horizontales.

Grad-CAM

Gradient-weighted Class Activation Mapping

Técnica de explicabilidad que genera un mapa de calor sobre la imagen mostrando qué zonas activaron más la predicción del modelo.

Ej: para una predicción 'gato', Grad-CAM muestra en rojo la zona donde estaba la cara del gato.

Normalización de Píxeles

Pixel Normalization

Proceso de escalar los valores de píxel del rango [0, 255] a [0, 1] o [-1, 1] antes de pasarlos a la red.

Ej: X_train / 255.0 en MNIST; rescaling por 1/127.5 - 1 en MobileNetV2.

Bounding Box

Bounding Box

Rectángulo que delimita la posición de un objeto detectado en una imagen, definido por sus coordenadas (x, y, ancho, alto).

Ej: YOLO devuelve bounding boxes para cada objeto que detecta en un frame de video.

YOLO

You Only Look Once (YOLO)

Arquitectura de detección de objetos en tiempo real que analiza la imagen una sola vez y detecta múltiples objetos simultáneamente.

Ej: YOLOv8n detecta 80 clases de objetos a 30+ FPS en una CPU normal.

Modelos de Lenguaje

LLM

Large Language Model (LLM)

Modelo de lenguaje entrenado con enormes cantidades de texto que aprende a generar, completar y razonar con lenguaje natural.

Ej: GPT-4, Llama 3, Gemini, Claude son LLMs. Se usan para chatbots, resumen de texto, generación de código.

Token

Token

Unidad básica de texto que procesa un LLM. Puede ser una palabra, parte de una palabra o un signo de puntuación.

Ej: 'Transfer Learning' se divide en tokens como ['Transfer', ' Learn', 'ing']. GPT-4 procesa hasta 128k tokens por prompt.

Ollama

Ollama

Herramienta open source que permite correr LLMs localmente en tu computadora, sin enviar datos a internet.

Ej: ollama run llama3 descarga y corre Llama 3 localmente, exponiendo una API en localhost:11434.

Prompt

Prompt

Instrucción o texto de entrada que le damos a un LLM para guiar su respuesta.

Ej: 'Resumí este artículo en 3 puntos clave' es un prompt. La calidad del prompt impacta directamente la calidad de la respuesta.

Embedding

Embedding / Representación vectorial

Transformación de texto (palabras, frases) en vectores numéricos que capturan su significado semántico.

Ej: 'rey' y 'reina' tienen embeddings cercanos en el espacio vectorial. 'perro' y 'gato' también.

Herramientas y Ecosistema

TensorFlow

TensorFlow

Framework de Deep Learning de Google. Permite construir, entrenar y desplegar modelos de redes neuronales. Keras es su API de alto nivel.

Ej: import tensorflow as tf → el import más común en los notebooks de esta diplomatura.

Keras

Keras

API de alto nivel incluida en TensorFlow que simplifica la construcción de redes neuronales con código legible y conciso.

Ej: keras.Sequential([layers.Dense(128), layers.Dense(10)]) define una red completa en 2 líneas.

PyTorch

PyTorch

Framework de Deep Learning de Meta, alternativa a TensorFlow. Preferido en investigación por su flexibilidad.

Ej: import torch → la base de modelos como Llama, Stable Diffusion y la mayoría de modelos de investigación.

Google Colab

Google Colaboratory (Colab)

Entorno de notebooks de Python en la nube con acceso gratuito a GPU. No requiere instalación local.

Ej: todos los notebooks de esta diplomatura corren en Colab. Accedé en colab.research.google.com.

Dataset

Dataset

Conjunto de datos usado para entrenar, validar o testear un modelo. Se divide típicamente en train (80%), validación (10%) y test (10%).

Ej: MNIST tiene 70.000 imágenes de dígitos. COCO tiene 330.000 imágenes con 80 categorías.

MNIST

Modified National Institute of Standards and Technology

Dataset de referencia con 70.000 imágenes de dígitos manuscritos (0-9) de 28x28 píxeles en escala de grises.

Ej: el 'Hola Mundo' del Deep Learning. Nuestra red densa alcanza ~98% de accuracy en MNIST.

HuggingFace

HuggingFace

Plataforma open source que centraliza miles de modelos pre-entrenados, datasets y demos de IA, listos para usar.

Ej: huggingface.co tiene modelos para texto, imagen, audio y video disponibles con unas pocas líneas de código.

¿Encontrás un término que falta?

Este glosario es un documento vivo. Si durante la cursada aparece un término nuevo que no está acá, lo agregamos.

Diplomatura en IA · UTN 2026
Módulo 2 — Deep Learning